

The Timetable Assistant

How an English sentence becomes campus data and scheduling rules
— architecture and code flow

Subsystem	Natural-language campus data entry & the timetable constraint registry
Entry point	<code>POST /api/assistant/interpret</code> → <code>/apply</code>
Core files	<code>app/tools/ops.py</code> · <code>app/api/assistant.py</code> · <code>app/tools/constraints.py</code> · <code>app/solver/timetable_model.py</code>
UI	<code>apps/web/src/app/constraints/page.tsx</code>
Model	<code>gemini-3.6-flash</code> via the provider-agnostic factory
Status	Implemented and verified live, 2026-08-28

The system never asks a language model to build a timetable. It asks it to **write down what should exist** — teachers, subjects, rooms, classes and rules — as typed operations, checked against the real database and shown to you before anything is saved. A CP-SAT solver then does the scheduling, deterministically. **Everything in this document follows from that one split.**

1 Why it is built this way

The obvious design — hand the timetable problem to the model and ask for a schedule — fails on three counts. It cannot prove the result is clash-free; it cannot explain why a request is impossible; and it produces a different answer every run. A CP-SAT solver does all three well, and cannot understand a sentence.

So the model is placed *upstream* of the solver, not in place of it. Its entire job is to translate one sentence into rows of a table. That job is small enough to constrain completely:

What the assistant does

- Creates subjects, teachers, rooms and classes.
- Decides who teaches what, for which class.
- Builds open-elective bands and their options.
- Writes, edits and removes scheduling rules.
- Regenerates the timetable when the plan implies it.

What still constrains it

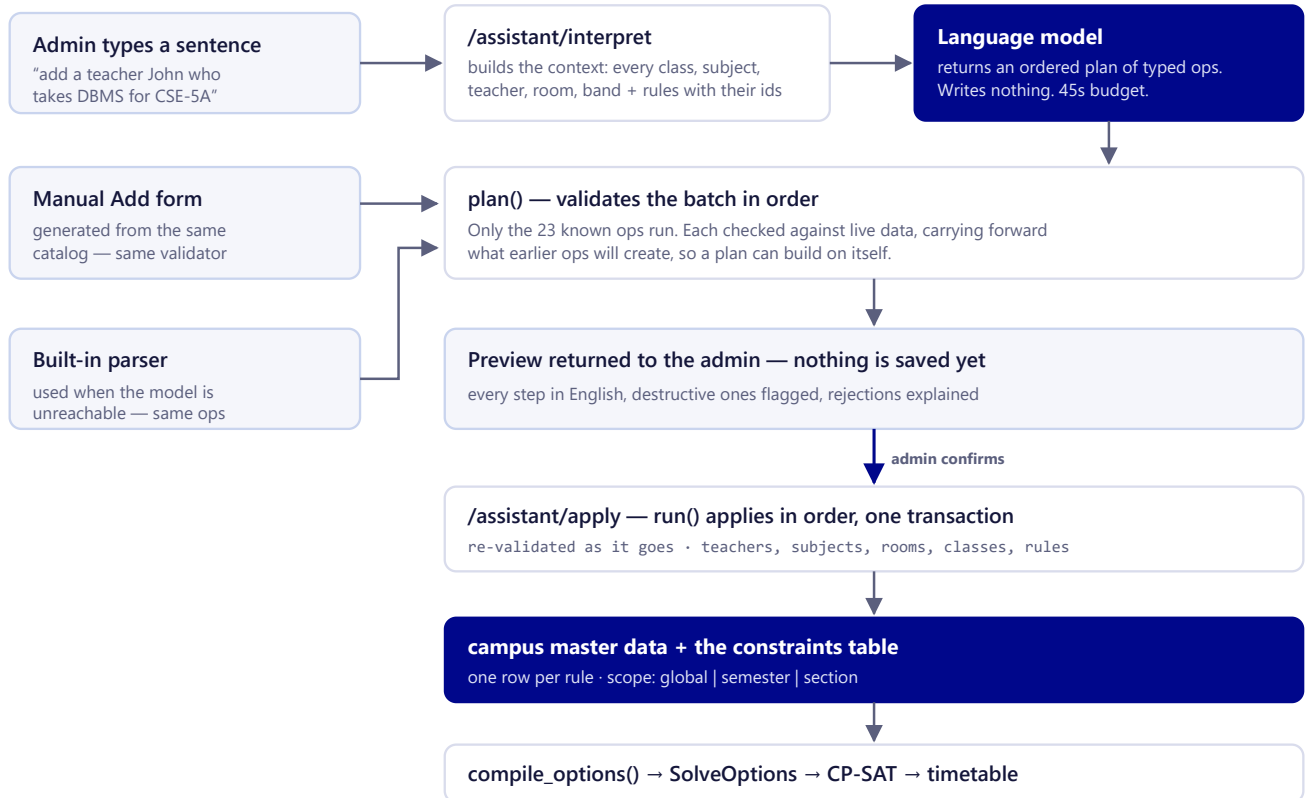
- A closed vocabulary of 23 operations — nothing else runs.
- Every operation validated against live data, twice.
- The whole plan previewed in English before it commits.
- No influence over *how* the solver satisfies anything.

A correction worth recording. Phase 2.4 *refused* to act on anything it could not already find — naming an unknown teacher was an error telling the admin to go and add them in the Setup forms. That was described as safety. It was not: it handed the repetitive work back to the person the system exists to save. Phase 2.5 keeps the typed, validated, previewed write and drops the refusal. **Creating the missing thing is the feature.**

The load-bearing idea. The model's output is not text that gets executed. It is a list of typed operations, each one validated before the preview and again as it runs. So the assistant can create freely without the blast radius that implies: an invented operation is refused, a parameter out of range is refused, and what does run is exactly what you read and approved.

2 Architecture at a glance

The model touches exactly one box in this pipeline, and it is not one that writes.



The confirm gate is the architectural centre: everything above it is a proposal, everything below it is committed state. The manual form and the offline parser join at `plan()`, so all three input paths share one set of guarantees — and the assistant can create data freely because nothing reaches the database without passing through it twice.

3 The vocabulary the assistant is held to

A closed set of operations, and within one of them, a closed set of rule kinds.

3.1 Twenty-three operations

`app/tools/ops.py` registers every action the assistant may take. Each `OpSpec` carries a validator (against live data), an English renderer for the preview, the write itself, and a `destructive` flag the UI colours red.

Area	Operations
Subjects	<code>add_subject</code> · <code>edit_subject</code> · <code>remove_subject</code>
Teachers	<code>add_teacher</code> · <code>edit_teacher</code> · <code>remove_teacher</code> · <code>set_teacher_subjects</code>
Rooms	<code>add_room</code> · <code>edit_room</code> · <code>remove_room</code>
Classes	<code>add_class</code> · <code>edit_class</code> · <code>remove_class</code>
Who teaches what	<code>assign_teacher</code>
Open electives	<code>add_elective_band</code> · <code>add_elective_option</code> · <code>remove_elective_band</code>
Rules	<code>add_rule</code> · <code>edit_rule</code> · <code>remove_rule</code> · <code>enable_rule</code> · <code>disable_rule</code>
Action	<code>generate_timetable</code>

3.2 Forward references — how one sentence builds on itself

"Add a teacher John who takes a new subject DBMS for CSE-5A" is three operations, and the second refers to something the first has not created yet. `plan()` walks the batch **in order** carrying a `PlanCtx`; each op registers what it will create through a `note` callback, so later ops can see it.

Rules needed more than a set of names. The rule validator resolves teachers and subjects to *objects with ids*, so a pending entity is inserted into its `Ctx` as a `_Stub` carrying only the attributes that validator reads — with a **negative id that is never persisted**. `run()` re-validates every op against the real database as it applies it, where the entity exists by then and has its real id.

```
# all four steps in one request – the rules refer to things that did not exist
add_class    CSE-2A, semester 2
add_rule     semester 2: at most 5 periods per day
add_teacher  Dr. Aditi Sharma
add_rule     Dr. Aditi Sharma is unavailable on FRI at P5, P6, P7
```

3.3 Implied fixups

Assigning a teacher to a subject they are not qualified for **adds the qualification** rather than erroring — the admin asked for the outcome, and chasing the prerequisite is exactly the busywork being removed. The preview says what it did:

```
CSE-5A CS501 → Dr. John Doe (also qualified them for it)
```

In the same spirit: a new teacher's email is derived from their name (`priya.menon@campus.edu`, uniquified), and subject codes are invented to match the conventions visible in the existing data.

3.4 Rule kinds: a catalog inside the vocabulary

A rule kind is not a string the code branches on in a dozen places. It is a `Kind` object that carries everything the system needs to know about that rule: where it may apply, how to check it, how to say it in English, and what the form

should look like.

app/tools/constraints.py:189

```
@dataclass
class Kind:
    key: str          # "half_day" - the stable machine name
    label: str        # "Half day" - shown in the UI
    help: str         # one line; also fed to the model
    scopes: tuple[str, ...] # global | semester | section
    fields: list[dict] # drives the form AND names the params
    validate: Callable[[Ctx, dict], dict] # normalise + reject
    describe: Callable[[dict], str] # render as English
    priority: str = "hard" # hard | soft (soft → objective term)
    unique_per_scope: bool = False # rewrite rather than stack
```

A registration reads as a single self-contained declaration:

app/tools/constraints.py:213

```
_register(Kind(
    key="half_day",
    label="Half day",
    help="A day ends after the given period; later periods are unused for that scope.",
    scopes=("global", "semester", "section"),
    fields=[_f("day", "day", "Day"), _f("last_period", "period", "Last teaching period")],
    validate=lambda ctx, p: {
        "day": _day(ctx, p.get("day")),
        "last_period": ... _period(ctx, p.get("last_period")),
    },
    describe=lambda p: f"{p['day']} ends after P{p['last_period']}",
))
```

Why this shape pays off. The catalog is serialised to JSON by `catalog_json()` and consumed by two very different clients: the React form renders inputs from `fields[].type`, and the model receives the same list as its menu of legal moves. Add a kind here and both gain it — *no prompt to rewrite, no form to edit*.

3.5 The 14 rule kinds

Kind	Scopes	Effect
half_day	all three	day ends after period N
no_same_subject_consecutive	all three	no adjacent same-subject theory periods
max_consecutive_teaching	all three	teacher run-length cap
subject_daily_max	all three	≤ N periods of one subject per day
max_periods_per_day	all three	cap on a class's day length
class_slot_ban	semester, section	class free at given days/periods
teacher_unavailable	global	teacher never scheduled then
teacher_max_daily	global	per-teacher daily cap
teacher_max_weekly	global	per-teacher weekly cap
teacher_max_consecutive	global	per-teacher run cap
subject_slot_ban	global	subject never at given slots
elective_fixed_slots	semester	pin an elective band to given slots
subject_preferred_periods	global	soft prefer these periods
avoid_gaps	global	soft penalise mid-day free periods

4 The registry — one row per rule

Before Phase 2.4 the rules lived in a request body and survived only as a JSON snapshot attached to a generated version: unlistable, uneditable, unexplainable. Now each is a row.

app/db/models.py — table `constraints`

Column	Holds
kind	a key from the catalog
scope_type / scope_value	<code>global</code> / <code>semester</code> + <code>"7"</code> / <code>section</code> + <code>"CSE-7A"</code>
params_json	the <i>normalised</i> params the validator produced
priority, weight	hard constraint, or soft objective term
enabled	toggled off without losing the rule
source	<code>seed</code> · <code>ui</code> · <code>llm</code> — provenance
origin_prompt	the sentence that created it, kept verbatim
description	the rendered English form, shown everywhere

Two columns worth dwelling on. `description` is generated once at write time by the kind's `describe()`, so the UI list, the flash message, the PDF export header and the model's own context all quote the identical wording — there is no second rendering path to drift. `origin_prompt` means a rule created six weeks ago can still tell you what someone actually asked for, which a params blob cannot.

Scope resolution

Rules are resolved **section** → **semester** → **global**, most specific wins. This is what lets a single solve produce a genuinely different week per semester: sem 3 on a six-period day with a Wednesday half-day, sem 7 on seven periods with a teacher run cap, in one model.

app/solver/timetable_model.py:195

```
def _resolve(opts: SolveOptions, sec: SectionIn) -> ScopeRules:
    """Effective rules for one section: global <- semester <- section."""
    sem = opts.semester_rules.get(sec.semester)
    own = opts.section_rules.get(sec.id)
    # half-days merge per day; every other field takes the narrowest non-None
```

5 Code flow of one prompt

What happens between pressing *Propose* and a rule existing.

1 The browser posts the sentence

`askAi()` — `constraints/page.tsx` — sends `{prompt}` to `/api/assistant/interpret` and puts the response in `preview` state. It commits nothing.

2 The server assembles the context the model will see

`_entity_context()` — `api/assistant.py` — builds one JSON object from live data — the whole campus, not just the rulebook:

```
{
  "classes": [{name, semester, dept, strength}],
  "subjects": [{code, name, semester, periods_per_week, lab}],
  "teachers": [{name, dept, max_hours_per_day, teaches:[codes]}],
  "rooms": [{name, type, capacity}],
  "elective_bands": [{name, semester, options:[...]}],
  "rule_catalog": [{kind, what, scopes, params}],
  "existing_rules": [{id, kind, scope, reads_as}],
  "days": [...], "periods": [...]
}
```

The highlighted line is what makes editing possible at all: because current rules arrive *with their ids*, "make that Wednesday half-day end at P3 instead" can be answered with an `edit_rule` on a specific row rather than a near-duplicate. Seeing the existing naming conventions is what lets the assistant invent a subject code that fits.

3 The model is asked for operations, not prose

The system prompt — `api/constraints.py:136` — states the split in its first lines and fixes the output shape:

```
# abridged
You are the Timetable Assistant of a college operations system.
You own everything about timetable data: subjects, teachers, rooms,
classes, who teaches what, open electives, and the scheduling rules.

THE MOST IMPORTANT RULE: if something the admin refers to does not exist
yet, CREATE IT. Never reply that a teacher, subject, room or class is
missing — that is the manual work you exist to eliminate.

Order matters. Emit operations in dependency order — create a subject
before a teacher who teaches it, the teacher before assigning them.

Fill in sensible detail rather than asking: follow the naming conventions
you can see; invent a subject code if none is given; omit email and it is
generated from the name.

Be conservative about destruction. Never remove something merely to
replace it if an edit_* op will do.

Reply with JSON only: {"ops": [...], "notes": "one short sentence"}
```

4 The call is bounded, then flattened to text

`_ask_llm()` — `api/assistant.py` — runs the invoke on a **daemon** thread with a 30-second budget, and passes the reply through `text_of()` before any JSON parsing. Section 7 explains why both details are load-bearing rather than incidental.

5 Every operation is validated — this is the safety boundary

`opsmod.plan()` — `tools/ops.py` — walks the batch **in order**, validating each op against live data and carrying forward what earlier ops will create (§6). A failed op is not dropped silently — it comes back with `ok: false` and a readable message, and the rest are still checked:

```
# real rejections
Room 'LT-401' already exists - edit it instead.
Ghost would teach unknown subject(s) NOPE99 - add those subjects in the same request.
Semester must be between 1 and 12, got 99.
'teleport_students' is not something I can do. Available: add_class, add_room, ...
```

6 A preview is returned — still nothing written

Each op carries its rendered `description`, and for an edit also `before`, so the diff is legible:

```
update    semester 5: FRI ends after P4
          was: semester 5: FRI ends after P5
```

7 You confirm; only then does anything commit

`applyAi()` posts the ops that validated to `/api/assistant/apply`, and `opsmod.run()` applies them **in order**, **in one transaction**, re-validating each against the real database as it goes. A failure is reported per-op.

8 The next generation compiles the rows

`compile_options()` — `tools/constraints.py:638` — turns enabled rows into a `SolveOptions`. `POST /api/timetable/generate` takes **no body at all**: what you can see on the page is, by construction, what generated the timetable.

6 Worked example

Tracing the exact sentence that exposed a missing capability, end to end.

“for semester 7 monday and tuesday 3rd hour should be open elective”

When this was first typed, the model had nothing legal to emit: an elective band's periods were chosen by the solver and no catalog kind could pin them. The fix was not a better prompt — it was a 14th kind. That is the honest test of this architecture: *a new capability is a catalog entry plus a solver constraint, and nothing else changes.*

Stage	Value
Model output	<code>{"op": "create", "kind": "elective_fixed_slots", "scope_type": "semester", "scope_value": "7", "params": {"days": ["MON", "TUE"], "periods": [3]}}</code>
Scope check	semester 7 exists (CSE-7A , CSE-7B) ✓
Band resolution	<code>_band()</code> finds one band in sem 7 → <code>OE-1</code> , so naming it was unnecessary
Normalised params	<code>{"band_id": 1, "band_name": "OE-1", "days": ["MON", "TUE"], "periods": [3]}</code>
Rendered	<i>semester 7: OE-1 runs at MON-P3, TUE-P3</i>
Compiled	<code>ElectivePin(group_id=1, slots=[("MON", 3), ("TUE", 3)])</code>
In CP-SAT	<code>m.Add(gv[(1, si)] == 1) for each pinned slot</code>
Result	both sem-7 classes sit in the band at MON-P3 and TUE-P3; the band's third weekly period floats to FRI-P2

Two design choices show up in that last row. Pinning is a *lower bound*, not an exact set — surplus band periods stay free for the solver, so you pin what you care about and leave the rest. And because a band is one shared object per semester, pinning it moves *every* class of that semester together, which is the entire point of an open elective.

When the request is impossible

Ask for four pinned slots on a three-period band and the precheck answers in the admin's vocabulary, before the solver runs at all:

OE-1 is fixed to 4 slot(s) but runs only 3 period(s)/week
– raise the band's periods per week or pin fewer slots.

This is why prechecks exist as a separate cheap pass: a bare "infeasible" from CP-SAT is useless to the person who typed the sentence. Where a precheck can't foresee the clash, the solver falls back on assumption literals to name the conflicting constraint *groups*.

7 Five failure modes, and the guard for each

Hazard	Guard
Invented rule kind teleport_students	<code>validate()</code> looks the kind up in <code>CATALOG</code> and raises listing every supported kind. Rejected in the preview; never stored.
Hallucinated entity CSE-9Z, "Dr. Strange"	Resolvers <code>_teacher</code> / <code>_subject</code> / <code>_band</code> and the scope check query live rows. In practice the model usually declines first — it is told the real names — and returns a note instead of an op.
Duplicate rules re-running a prompt	Two layers. Kinds marked <code>unique_per_scope</code> rewrite their row. Otherwise <code>create()</code> compares <i>normalised</i> params, so <code>MON,TUE/P3</code> and <code>TUE,MON/"P3"</code> are recognised as one rule and the existing row is reused.
Provider stalls no error, no response	30-second budget on a daemon thread. LangChain offers no cancellation, so a timed-out worker is abandoned — and a <code>ThreadPoolExecutor</code> worker, being non-daemon, would block interpreter exit and uvicorn's reload. Falls back to the built-in parser.
Content blocks, not text	<code>text_of()</code> in <code>app/core/llm.py</code> . Newer providers return <code>content</code> as a list of typed blocks; <code>str()</code> -ing that yields a Python repr, which is not JSON. Every caller now goes through it.

Field note worth keeping. The symptom “the language model could not be reached” was traced not to the key but to the *model*: `gemini-flash-latest` is listed by `GET /v1beta/models` yet hangs forever on `generateContent` — no error, no disconnect. Listing models with the same key returns `200`, which isolates auth from any single model in one request. `.env` is pinned to `gemini-3.6-flash`.

When there is no model at all

`_fallback_ops()` — `api/constraints.py:201` — is a small deterministic parser covering the common phrasings. It emits *the same op dictionaries* and passes through *the same validator*, so the offline path is a narrower front door, not a second system. It handles the scope-digit trap you would hit immediately otherwise:

```
# "CSE-7A: at most 5 periods a day" - the 7 in the class name is not the limit
rest = re.sub(r"(?sem|semester)s*d+", " ", p)
if sec:
    rest = rest.replace(sec.lower(), "")
```

8 From a row to a scheduled period

`compile_options()` is a single dispatch over enabled rows, sorting each into the structure the solver reads. Scoped rules land in `semester_rules` or `section_rules`; targeted ones become list entries:

app/tools/constraints.py:638

```
elif c.kind == "teacher_unavailable":
    opts.bans.append(SlotBan(target="teacher", ref_id=p["teacher_id"],
                             days=p["days"], periods=p["periods"], label=c.description))
elif c.kind == "elective_fixed_slots":
    opts.elective_pins.append(
        ElectivePin(
            group_id=p["band_id"],
            slots=[(d, n) for d in p["days"] for n in p["periods"]]))
```

The solver then turns each into CP-SAT constraints, every group behind its own assumption literal so an infeasible combination can still name the culprit:

Compiled form	CP-SAT
ScopeRules.half_days	slots outside the day are pinned <code>x == 0</code> for that section
SlotBan(teacher)	<code>sum(busy at slot) == 0</code> for that teacher
SlotBan(subject)	<code>x[sec, subj, slot] == 0</code> across sections
ElectivePin	<code>gv[band, slot] == 1</code>
max_consecutive_teaching	in each window of k+1 adjacent slots, $\leq k$ taught
PreferredPeriods soft	weighted penalty per placement outside the preferred set
avoid_gaps soft	prefix/suffix busy flags mark any idle period inside the teaching day

Hard vs soft is a property of the row, not of the code path. A hard rule becomes a constraint the solver must satisfy or declare infeasible; a soft one becomes a weighted term in the objective it merely prefers to satisfy. Both come from the same table, and the UI marks soft rules so you know which are negotiable.

9 Where everything lives

File	Responsibility
app/tools/ops.py	the 23 operations, <code>plan()</code> / <code>run()</code> , forward references
app/api/assistant.py	the model call, the system prompt, preview/apply, offline parser
app/tools/constraints.py	rule catalog, validators, CRUD, <code>compile_options()</code>
app/api/constraints.py	the rulebook's own CRUD (NL routes delegate to the assistant)
app/solver/timetable_model.py	CP-SAT model, scope resolution, prechecks
app/tools/timetable.py	loads solver input, stores versions, <code>current_options()</code>
app/core/llm.py	provider-agnostic factory, <code>text_of()</code>
app/db/models.py	<code>Constraint</code> , <code>ElectiveGroup</code> , <code>TeachingAssignment</code>
apps/web/.../constraints/page.tsx	rules list, Ask-AI confirm flow, catalog-driven form

10 Extending it

A twenty-fourth operation

Register an `OpSpec` in `OPS` — validate against `PlanCtx`, describe in English, apply — and list it in `_OP_REFERENCE` in `app/api/assistant.py`. If it creates an entity that later ops may reference, give it a `note` callback so forward references keep working.

A fifteenth rule kind

Three edits, in one direction, with no prompt engineering:

1. **Register a** `Kind` in `CATALOG` — scopes, form fields, validator, renderer.
2. **Handle it in** `compile_options()` — map the row onto a `SolveOptions` field.
3. **Enforce it in the solver**, behind its own assumption literal so infeasibility stays explainable.

The form and the model's prompt update themselves, because both are generated from the catalog. If the new kind's meaning depends on its scope — as `elective_fixed_slots` does, resolving its band within the scope semester — read the semester from the reserved `SCOPE_SEMESTER` key that `validate()` injects and strips again.

One deliberate piece of debt. `SCOPE_SEMESTER` is a reserved dictionary key rather than a proper argument on `Kind.validate`, because exactly one kind needs it today and changing the signature would touch all fourteen validators. If a second such kind appears, that trade flips — promote it to an argument.

Smart Campus Agent System · Phase 2.4 · constraint prompting. Companion documents: `docs/10-PHASE2.5-PLAN.md` and `docs/09-PHASE2.4-PLAN.md` (full specs), `docs/02-ARCHITECTURE.md` (system design), `docs/04-ROADMAP.md` (status). Line references are against the working tree of 2026-08-28.